

SkillSwap — Phase 3

Application & UI/UX Design

Phases 3.1 to 3.9

This document defines the application architecture, user roles, user flows, screen structure, wireframes, UI/UX direction, navigation, responsive behavior, and validation/security plan for the SkillSwap peer-to-peer skill exchange platform.

3.1 — System Architecture

Overview

SkillSwap will use a three-layer web architecture: a React frontend, a Node.js/Express backend API, and a MySQL relational database.

Architecture

Browser / React UI
↓ HTTP/HTTPS + REST API
Node.js + Express Backend
↓ SQL / database driver
MySQL Database

Frontend responsibilities

User interface, navigation, forms, client-side validation, loading/error states, authentication state, skill search, profiles, requests, sessions, and reviews.

Backend responsibilities

REST API endpoints, authentication and authorization, business rules, server-side validation, CRUD operations, database access, and error handling.

Database responsibilities

Persistent storage for users, categories, skills, user skills, learning requests, swap requests, sessions, and reviews.

Recommended project structure

```
skillswap/  
■■■■ frontend/  
■ ■■■■ src/  
■ ■ ■■■■ components/  
■ ■ ■■■■ pages/  
■ ■ ■■■■ layouts/  
■ ■ ■■■■ services/  
■ ■ ■■■■ hooks/  
■ ■ ■■■■ utils/  
■ ■■■■ package.json  
■■■■ backend/  
■ ■■■■ controllers/  
■ ■■■■ routes/  
■ ■■■■ models/  
■ ■■■■ middleware/
```

- ■■■ services/
- ■■■ config/
- ■■■ server.js
- database/
- skillswap.sql

3.2 — User Roles & Access

Normal User

Register, log in, manage profile, add teaching skills, browse/search skills, create learning requests, send and respond to swap requests, schedule sessions, view sessions, and submit reviews.

Administrator

Access an administrative dashboard, manage users, categories and skills, monitor requests and sessions, and handle inappropriate or invalid data.

Access principle

Every protected action must be checked on the backend. The UI may hide unavailable actions, but authorization must never depend only on frontend controls.

3.3 — User Flow

Main flow

Start

- Register
- Login
- Create / complete profile
- Add skills
- Browse or search for a skill
- View another user's profile
- Send skill-swap request
- Receiver accepts or rejects
- If accepted: schedule session
- Attend / complete session
- Submit review

Alternative request flow

Pending → Accepted → Scheduled → Completed → Reviewed

Rejected flow

Pending → Rejected. The user can search for another skill partner.

3.4 — Page & Screen Structure

Public pages

Home, About, Browse Skills, Login, Register.

User pages

Dashboard, Profile, My Skills, Find Skills, Learning Requests, Swap Requests, Sessions, Reviews, Settings.

Admin pages

Admin Dashboard, Manage Users, Manage Categories, Manage Skills, Manage Requests, Manage Sessions.

Primary navigation

Desktop: Logo, Home, Find Skills, Requests, Sessions, Profile. Mobile: collapsible menu containing the same main destinations.

3.5 — Wireframes

Home page

[SkillSwap Logo] Home | Find Skills | Login

Learn a Skill.

Share a Skill.

Connect with people and exchange skills.

[Find a Skill] [Start Teaching]

Popular Skills | Categories | How It Works

Featured Users | Benefits | Call to Action

Footer

Dashboard

Sidebar: Dashboard / Profile / My Skills / Find Skills / Requests / Sessions

Main:

Welcome, User!

[Skills You Teach] [Learning Requests] [Recent Requests]

[Upcoming Sessions]

Find Skills

Search: [What do you want to learn?] [Search]

Filters: Category | Level | Location

User Card

Name | Skills | Rating

[View Profile] [Request Swap]

Profile

[Profile Photo] Name Location

Bio

Skills I Teach

[JavaScript - Advanced] [React - Intermediate]

Rating

[Request Skill Swap]

Swap request

I can teach: [Skill ▼]

I want to learn: [Skill ▼]

Message: [.....]

[Send Request]

Session

Skill Exchange Session

Users: Sarah ↔ Alex

Skills: JavaScript ↔ Photoshop

Date | Time | Duration

Status: Scheduled

Review

How was your session?



Comment: [.....]

[Submit Review]

3.6 — UI/UX Design

Design direction

Use a clean, modern education/community visual style with clear cards, readable typography, skill badges, consistent buttons, accessible forms, and responsive layouts.

Core components

Navbar, sidebar, buttons, input fields, select fields, skill cards, user profile cards, request cards, session cards, review cards, modal/dialog, toast/alert, loading indicator, empty state, pagination where needed.

Interaction principles

Users should always know what action is available, what state an item is in, and what happens after submitting a form. Use confirmation messages for destructive actions.

Status design

Use clear labels for PENDING, ACCEPTED, REJECTED, CANCELLED, SCHEDULED, COMPLETED, and ACTIVE states.

3.7 — Navigation Design

Desktop navigation

Logo | Home | Find Skills | Requests | Sessions | Profile | Logout.

Mobile navigation

Hamburger menu containing Home, Find Skills, My Skills, Requests, Sessions, Profile, and Logout.

Admin navigation

Dashboard | Users | Categories | Skills | Requests | Sessions | Logout.

Navigation rule

Navigation should be consistent across pages and the current page should be visually identifiable.

3.8 — Responsive Design

Breakpoints

The interface should adapt to desktop, laptop, tablet, and mobile screen sizes.

Desktop

Use sidebar + main content, multi-column cards, full navigation, and wider forms.

Tablet

Reduce columns and allow cards to wrap; keep primary actions visible.

Mobile

Use a collapsible navigation menu, single-column layouts, stacked cards, full-width primary buttons, and touch-friendly controls.

Responsive rule

No horizontal scrolling should be required for normal application content.

3.9 — Validation, Security & Error Handling

Client-side validation

Required fields, valid email format, password rules, minimum/maximum text lengths, valid selections, rating range 1–5, positive session duration, and clear inline error messages.

Server-side validation

Repeat all important validation on the backend. Never trust values sent by the browser.

Authentication

Use secure password hashing, protected routes, session/token management, logout, and authentication checks for private resources.

Authorization

Check user ownership and role permissions before update/delete operations. Admin-only endpoints must verify the administrator role.

Database security

Use parameterized queries or a safe ORM/query builder, foreign keys, UNIQUE constraints, CHECK constraints, and controlled delete/update behavior.

Input security

Sanitize or safely render user-generated text, validate uploaded content if uploads are later added, and avoid exposing passwords or sensitive fields in API responses.

Error handling

Provide friendly UI messages such as 'Unable to send request. Please try again.' while keeping detailed technical errors in server logs.

Example validation rules

Rating must be 1–5; requester and receiver must be different users; offered and requested skills should be valid; duplicate user-skill pairs are not allowed.

Phase 3 Completion Checklist

Phase	Deliverable	Status
3.1	System Architecture	Complete
3.2	User Roles & Access	Complete
3.3	User Flow	Complete
3.4	Page & Screen Structure	Complete
3.5	Wireframes	Complete
3.6	UI/UX Design	Complete
3.7	Navigation Design	Complete

Phase	Deliverable	Status
3.8	Responsive Design	Complete
3.9	Validation, Security & Error Handling	Complete